

Neural networks: training and optimization

STAT 4710

November 11, 2025

Where we are

- ✓ **Unit 1:** R for data mining
- ✓ **Unit 2:** Prediction fundamentals
- ✓ **Unit 3:** Regression-based methods
- ✓ **Unit 4:** Tree-based methods
- Unit 5:** Deep learning

Lecture 1: Deep learning preliminaries

Lecture 2: Training and optimization of NNs

Lecture 3: Deep learning for images

Lecture 4: Deep learning for text

Lecture 5: Unit review and quiz in class

Plan for Unit 5: concepts & hands-on

Lecture 1: Deep learning preliminaries

- Predictive models as graphs
- Training via optimization: gradient descent
- Hands-on: simple NN on MNIST in R torch

Lecture 2: Neural networks

- Multi-layer predictive models
- Stochastic gradient descent
- Hands-on: multi-layer NN for MNIST in R torch with newly learned training options

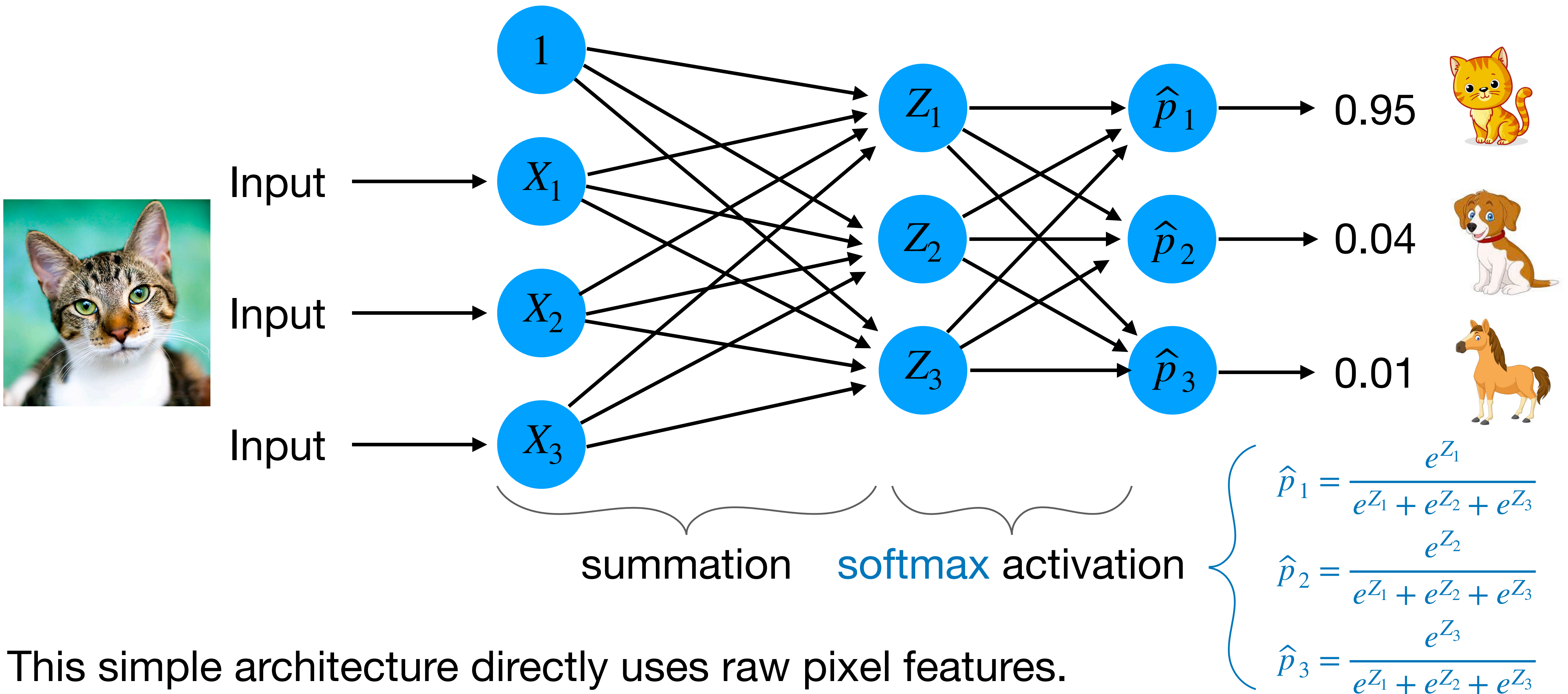
Lecture 3: Deep learning for images

- Image classification
- Convolutional neural networks
- Hands-on: CNN for MNIST in R torch

Lecture 4: Deep learning for text

- Document classification
- Recurrent neural networks and transformers

Recall: Multi-class logistic model



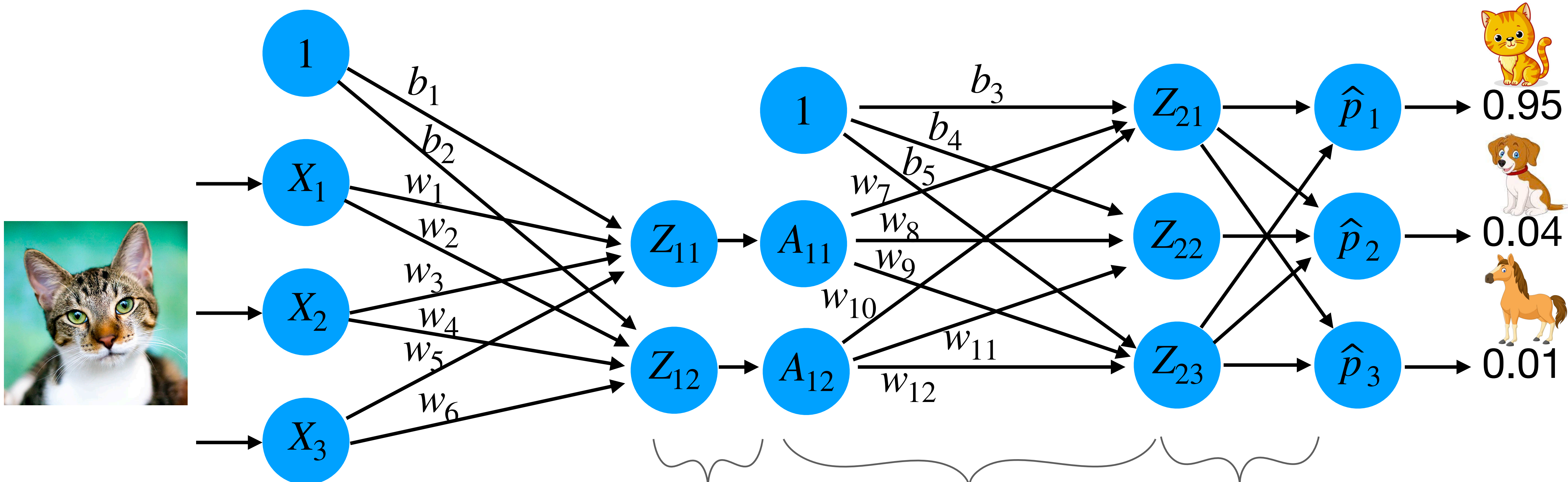
This simple architecture directly uses raw pixel features.
Intuitively, a good classifier would pick up on cat's eyes, ears, etc.

One-hidden-layer neural network

Input layer

Hidden layer

Output layer



summation

E.g., $Z_{12} = b_2 + w_2X_1 + w_4X_2 + w_6X_3$

ReLU

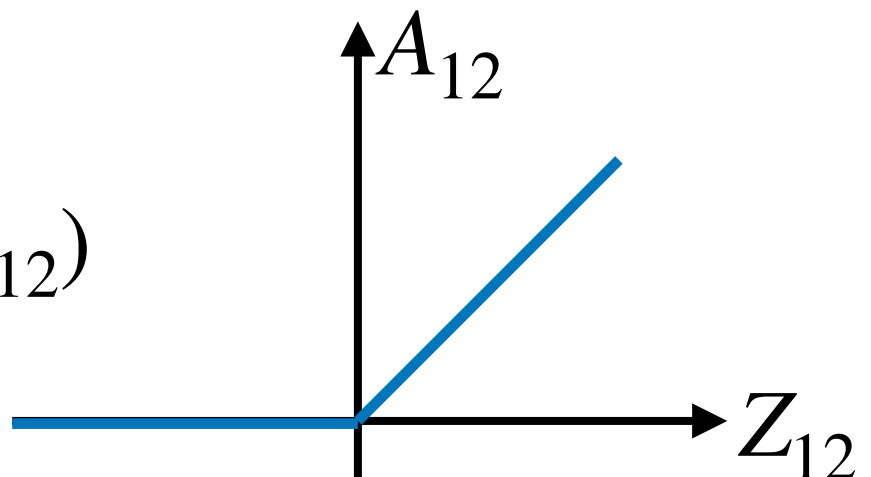
activation

E.g., $A_{12} = \max(0, Z_{12})$

summation

softmax

activation



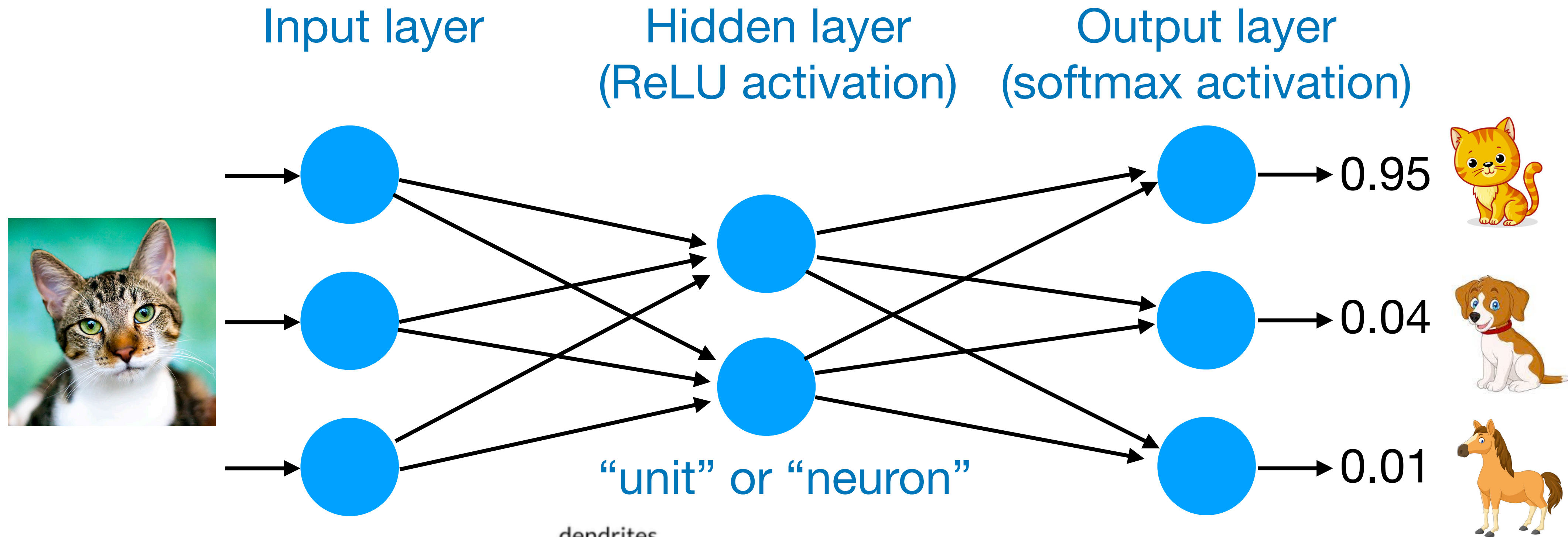
Terminology:

b_j : biases

w_j : weights

One-hidden-layer neural network

Concise representation

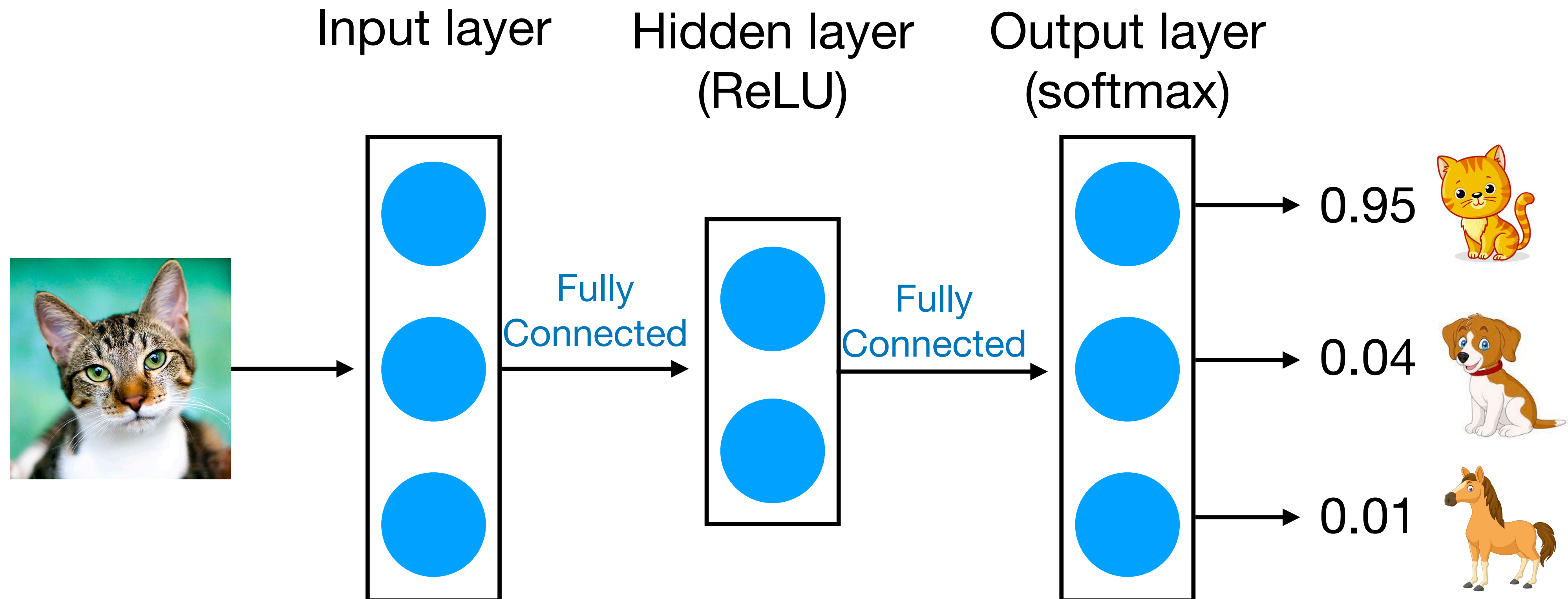


Notes:

- Bias terms left implicit.
- Fully connected architecture.
- Hidden layer: learned features.

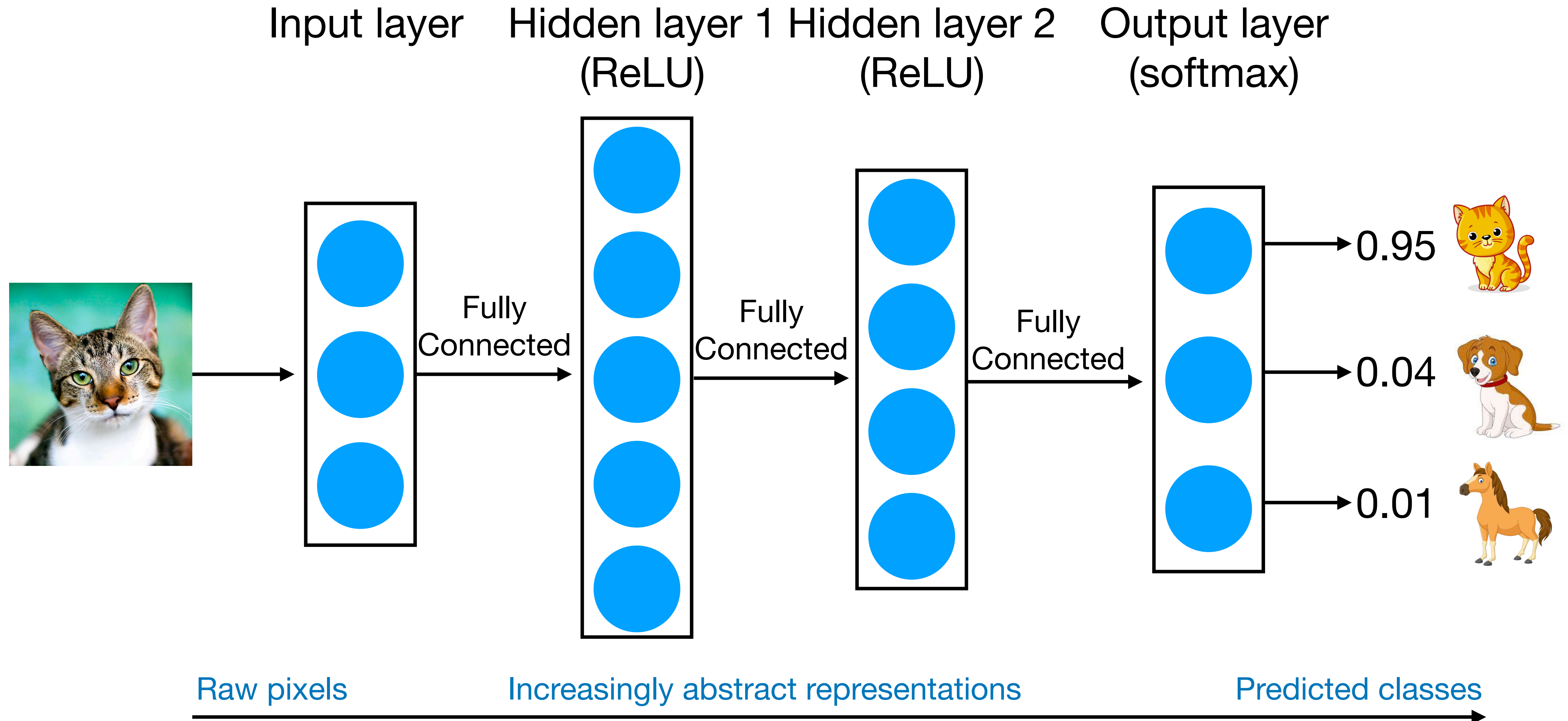
One-hidden-layer neural network

Even more concise representation



Number of parameters in FC layer =
 $(\# \text{ units in previous layer} + 1) \times (\# \text{ units in this layer})$

Multi-layer fully connected neural networks



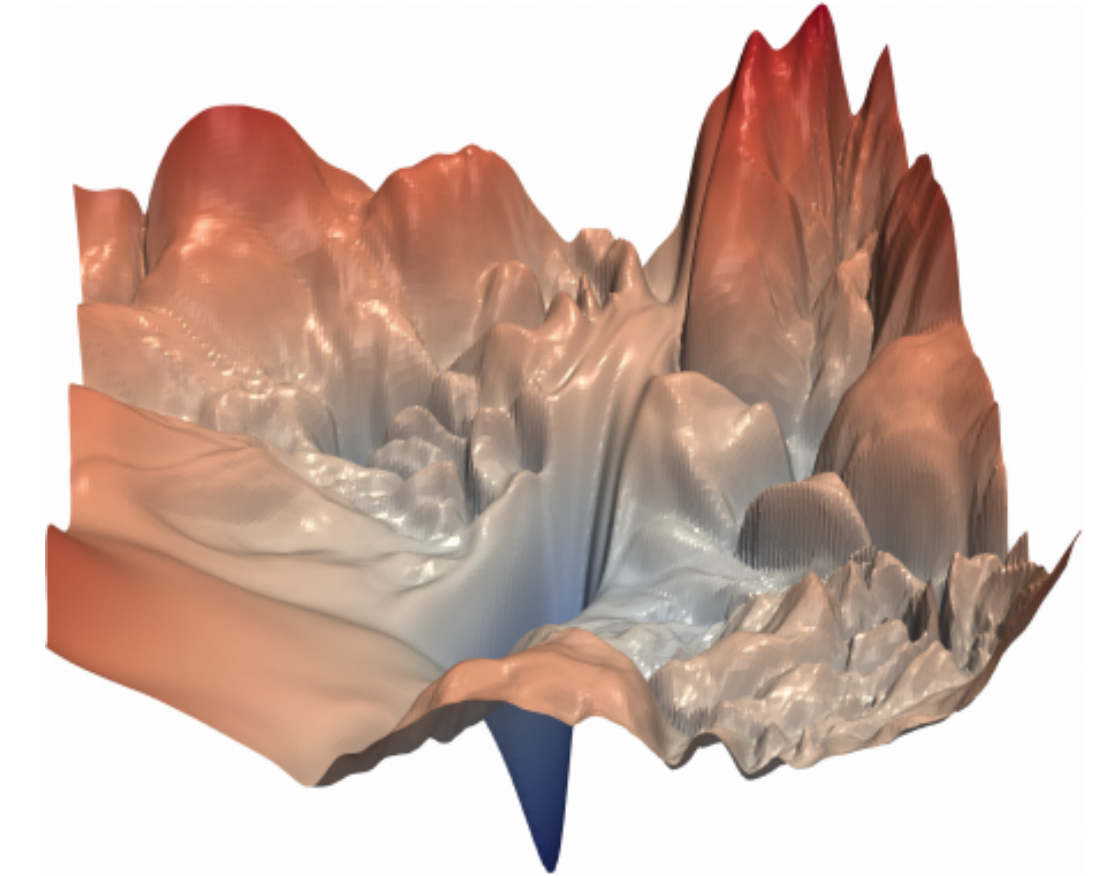
Training deep learning models

Training a neural network amounts to solving our usual optimization problem:

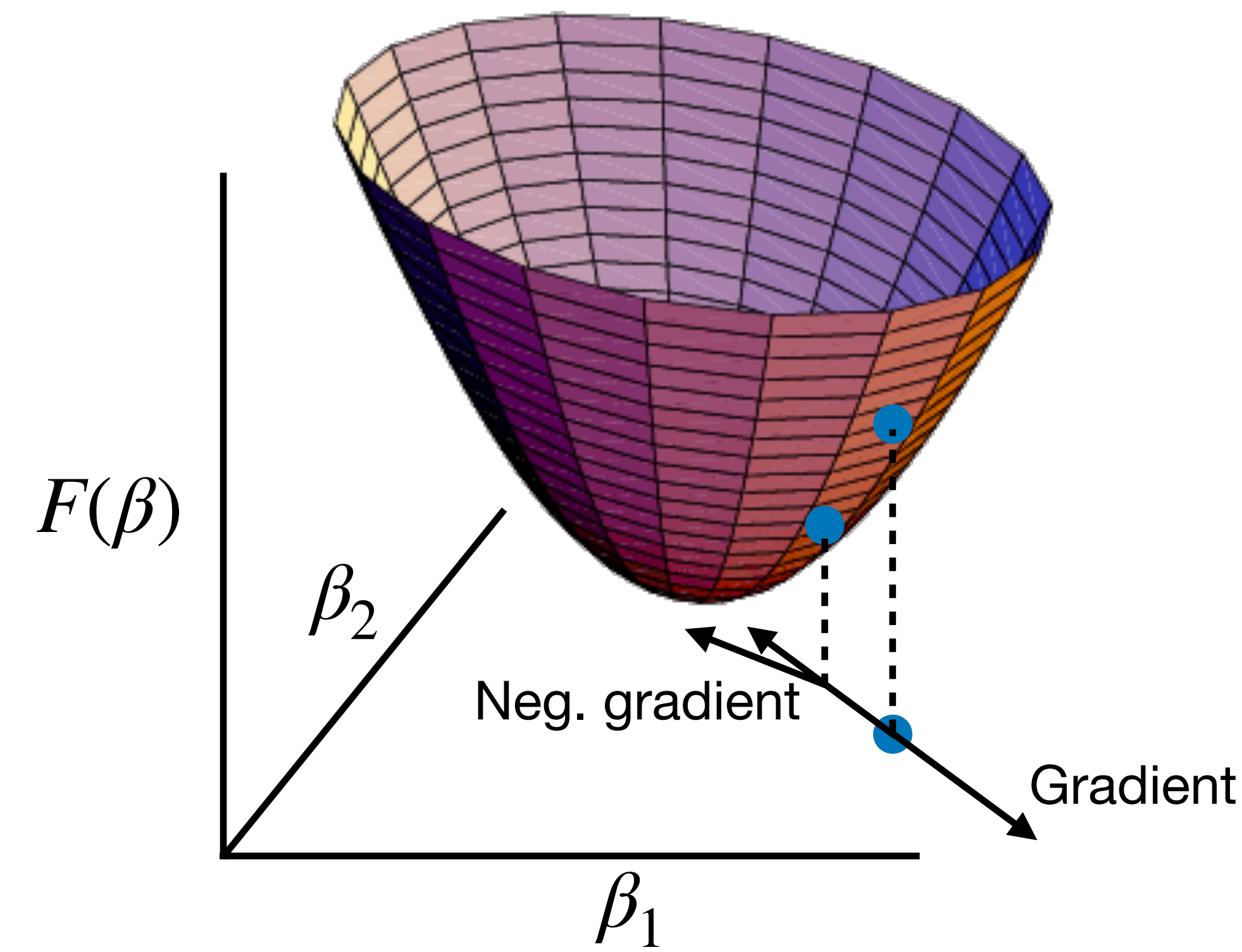
$$\hat{\beta} = \arg \min_{\beta} \frac{1}{n} \sum_{i=1}^n L(Y_i, f_{\beta}(X_i)) + \lambda \cdot \text{penalty}(\beta) = \arg \min_{\beta} F(\beta).$$

Neural networks have non-convex objective functions, discussed in the previous lecture.

In practice, deep learning models are trained using **stochastic gradient descent**, which is a variant of gradient descent.



<https://arxiv.org/abs/1712.09913>



The computational cost of gradient descent

computational cost = (cost of computing the gradient) $\times$ (number of iterations)

Cost of computing the gradient has to do with the size of the training data:

$$\text{If } F(\beta) = \frac{1}{n} \sum_{i=1}^n L(Y_i, f_{\beta}(X_i)), \text{ then } \nabla F(\beta) = \frac{1}{n} \sum_{i=1}^n \nabla L(Y_i, f_{\beta}(X_i)).$$

Each evaluation of the gradient requires a pass through the entire training data (millions for modern datasets).

This can make gradient descent prohibitively expensive.

Stochastic gradient descent addresses the computation issue of GD.

Stochastic gradient descent

Use subset $S \subseteq \{1, \dots, n\}$ (mini-batch) of observations to approximate gradient:

$$\nabla F(\beta) = \frac{1}{n} \sum_{i=1}^n \nabla L(Y_i, f_{\beta}(X_i)) \approx \frac{1}{|S|} \sum_{i \in S} \nabla L(Y_i, f_{\beta}(X_i)).$$

1. Choose some initial value of β .
2. Randomly assign observations to mini-batches $S_1, \dots, S_M$ of a certain size
3. Repeat until convergence:

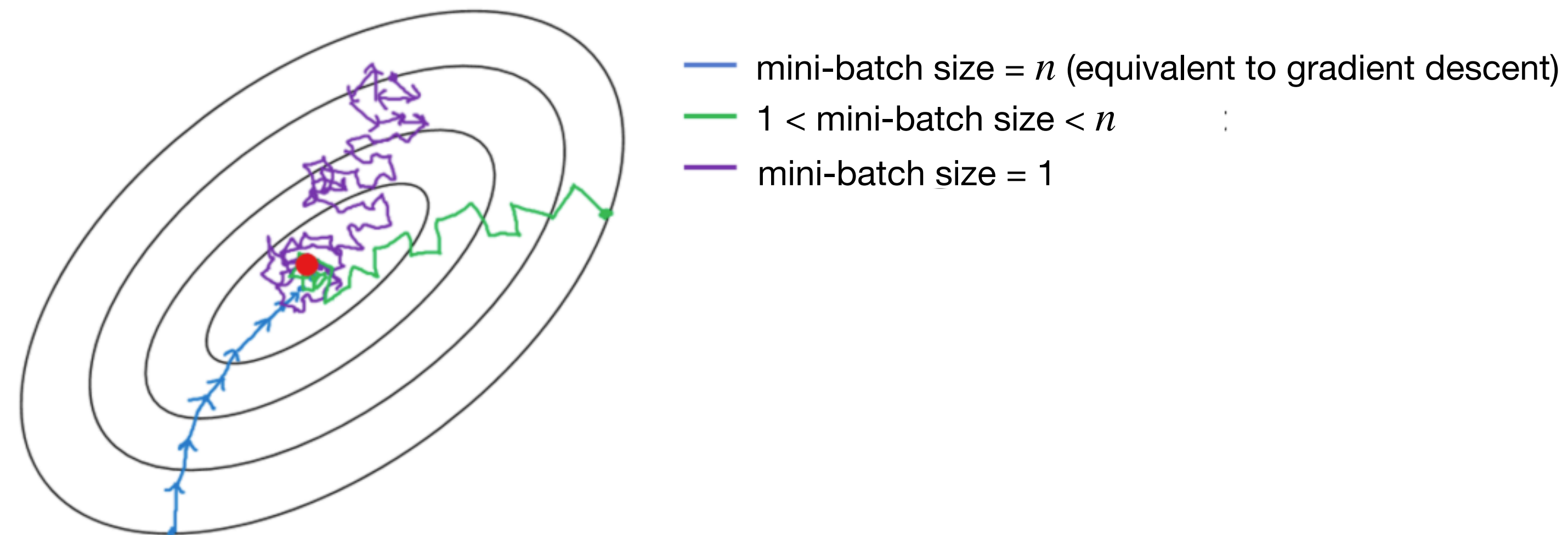
- For $m = 1, \dots, M$, update $\beta \leftarrow \beta - \gamma \cdot \frac{1}{|S_m|} \sum_{i \in S_m} \nabla L(Y_i, f_{\beta}(X_i))$.

One “epoch”

Back propagation: An efficient algorithm to compute $\nabla L(Y_i, f_{\beta}(X_i))$

Behavior of SGD varies with batch size

Stochastic gradient descent wobbles toward decreasing values of the objective:



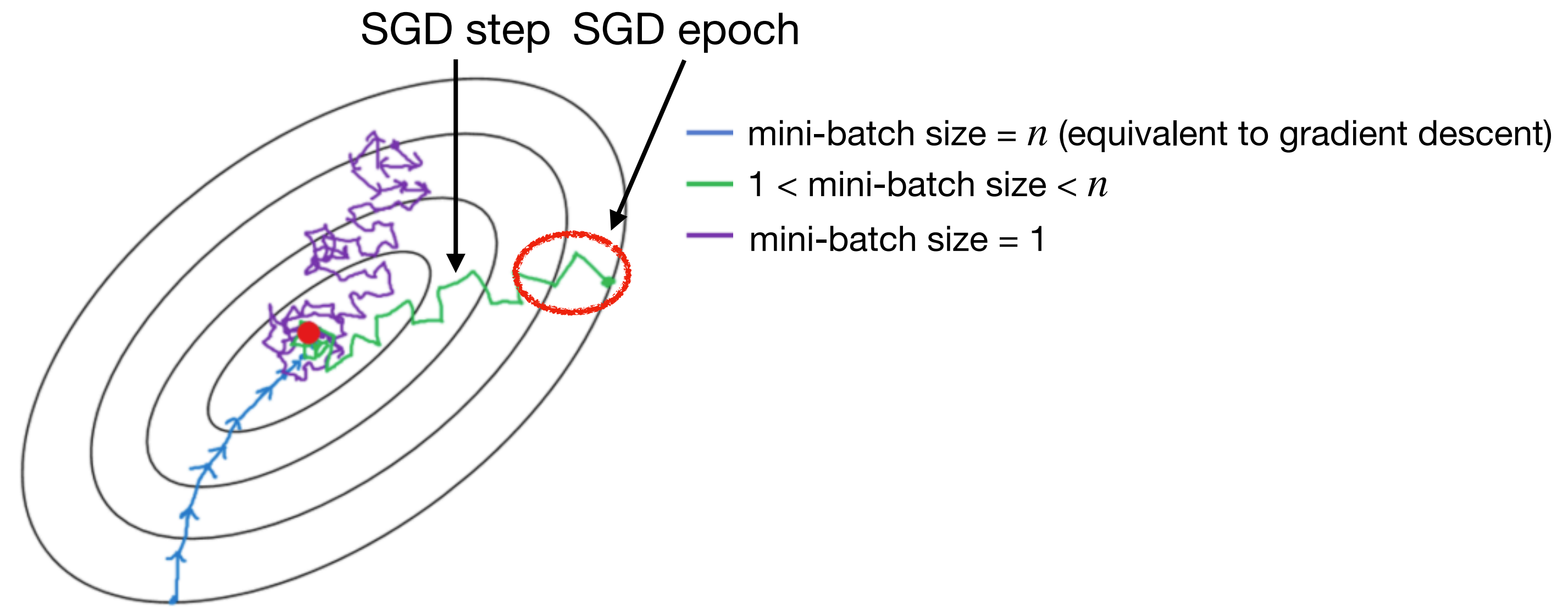
Source: <https://medium.com/analytics-vidhya/gradient-descent-vs-stochastic-gd-vs-mini-batch-sgd-fbd3a2cb4ba4>

The smaller the mini-batch, the cheaper and more wobbly each step is; Intermediate mini-batch sizes tend to work well, e.g. mini-batch size = 32.

Bonus: The extra randomness sometimes allows SGD to wobble past local minima.

Stochastic gradient descent step versus epoch

An epoch consists of SGD steps that cycle through all of the observations once.

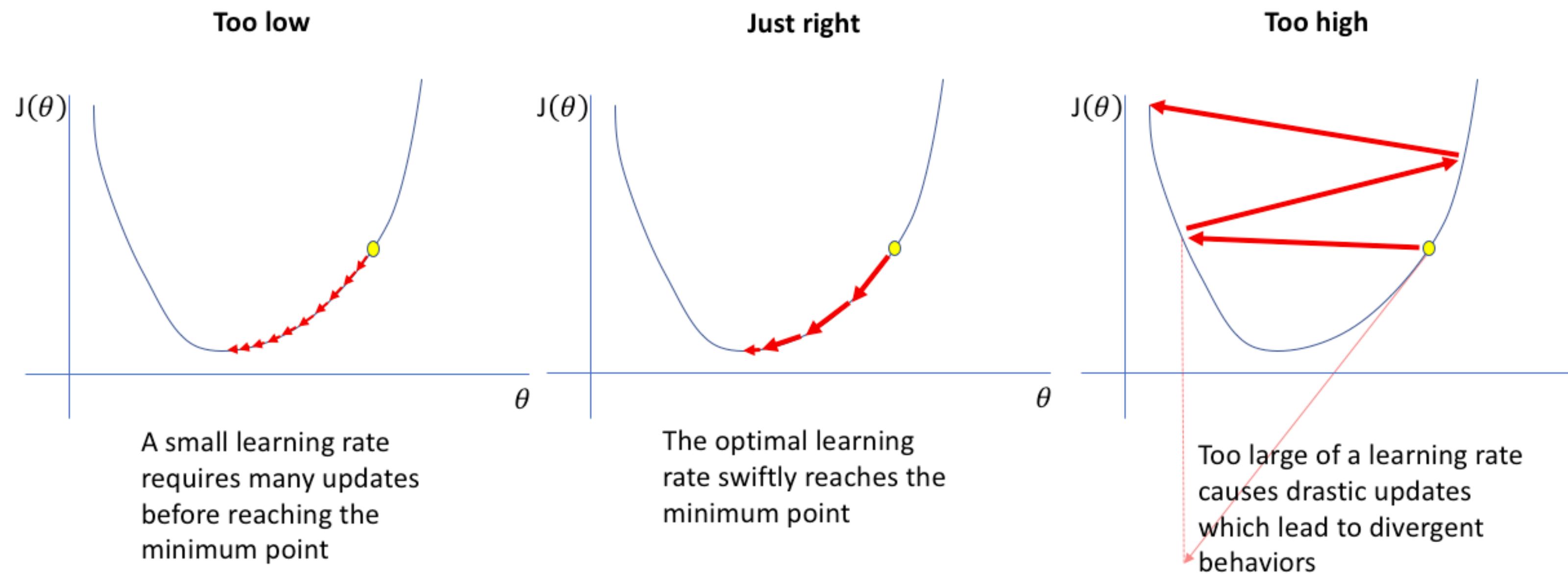


Source: <https://medium.com/analytics-vidhya/gradient-descent-vs-stochastic-gd-vs-mini-batch-sgd-fbd3a2cb4ba4>

E.g. $n = 9$, mini-batch size 3. Then each epoch consists of three SGD steps.

Observation	1	2	3	4	5	6	7	8	9	1	2	3	4	5	6	7	8	9
Epoch	Epoch 1									Epoch 2								
SGD step (mini-batch size 1)	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
SGD step (mini-batch size 3)	Step 1			Step 2			Step 3			Step 4			Step 5			Step 6		
Gradient step	Step 1									Step 2								

Behavior of SGD varies with learning rate



Source: <https://www.jeremyjordan.me/nn-learning-rate/>

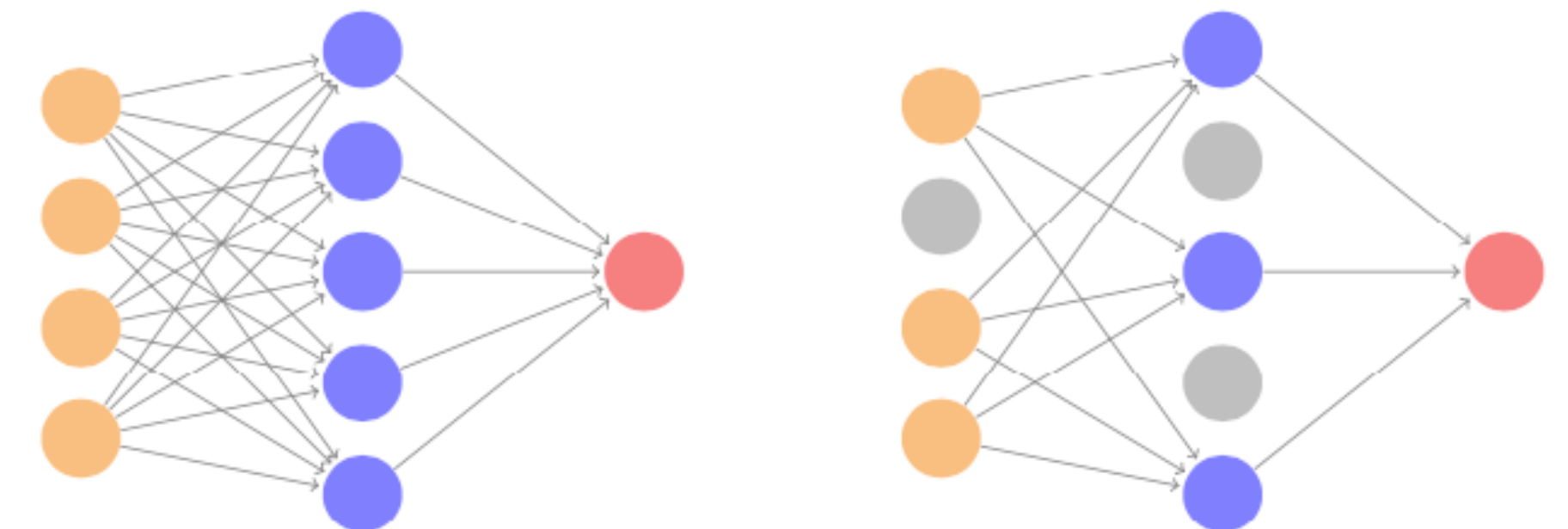
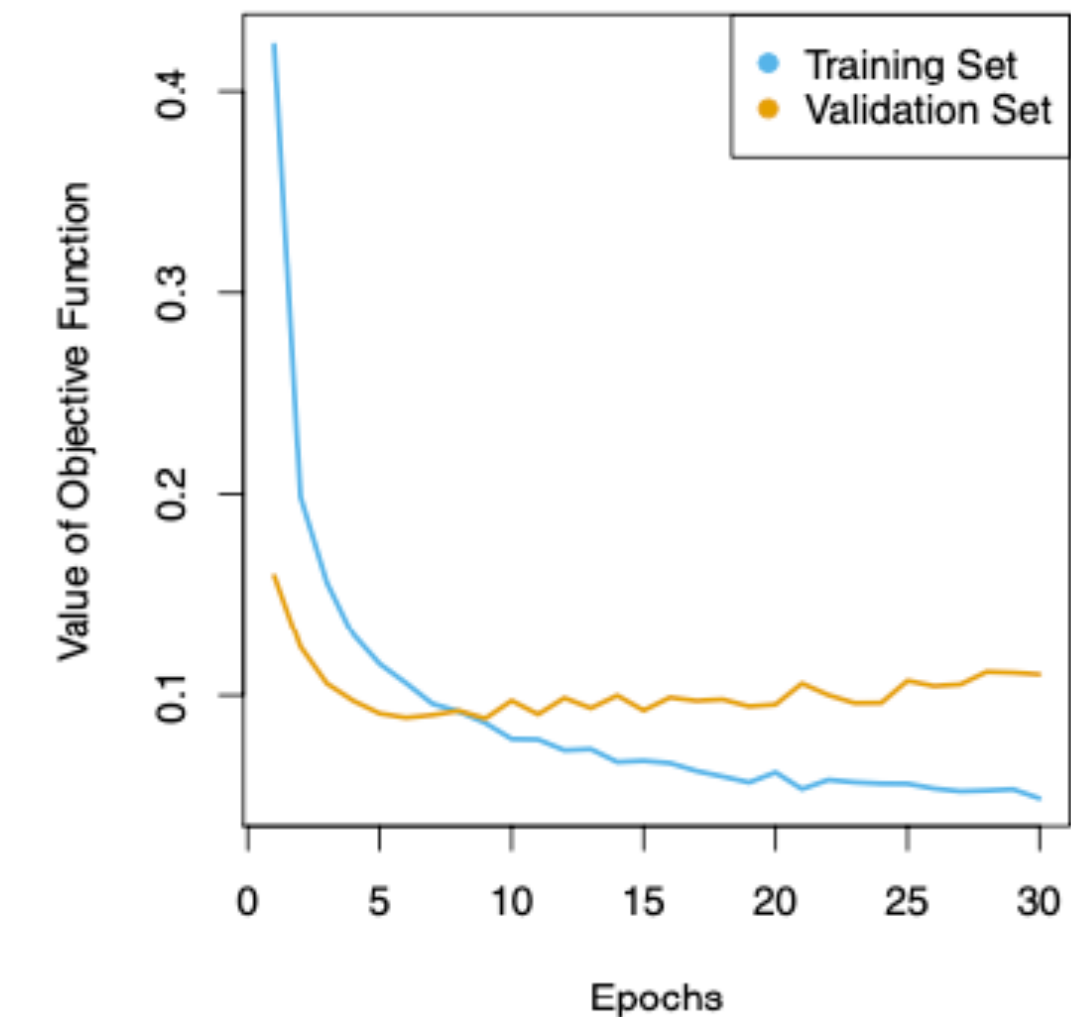
- Setting the learning rate is more of an art than a science; might need to try a few values to get a good one.
- Especially for non-convex optimization, people come up with clever strategies like shrinking learning rates, cycling learning rates, adaptive learning rates, etc. (RMSprop, Adam, AdaGrad, AdaDelta, ...)

Regularization in deep learning models

Explicit regularization via penalization. Ridge regression penalty is the most common; this kind of penalization is known as **weight decay**.

Implicit regularization: Other techniques to control complexity of the model.

- **Early stopping:** Do not run SGD until convergence; rather, stop SGD when validation error starts to increase.
- **Dropout:** At each SGD iteration, remove a randomly selected set of nodes from the network (analogous to sub-sampling features for random forests).



Model complexity and tuning in deep learning

Deep learning has many tuning parameters:

- Network architecture (number of layers, units per layer)
- Optimization algorithm used (RMSprop, Adam, etc.)
- Weight decay parameter, dropout rate, number of SGD iterations

More complex models arise from bigger networks, less weight decay, less dropout, more SGD iterations.

Deep learning performance can be sensitive to these parameters; there is no standard way to tune neural networks; tuning is computationally expensive.

For computational savings, **validation sets** are typically employed instead of cross-validation to tune neural networks.

Summary

- Adding hidden layers to logistic regression gives multilayer neural networks.
- Multilayer NNs build increasingly complex representations of the input data (feature learning).
- Training techniques of neural networks
 - Stochastic gradient descent uses noisy approximations to the gradient
 - Much faster than gradient descent, and may escape local minima
 - Explicit and implicit regularization are used to control model complexity
 - Explicit reg. includes weight decay.
 - Implicit reg. includes early stopping and dropout.
- NNs come with many tuning parameters, which often require considerable effort to tune.
- Thanks to powerful toolboxes, we can now easily build our NNs with customized options.

Hands-on: neural networks in torch

- We'll now build on our previous lecture to create multi-layer NNs.
- After the programming exercise, you will know how to
 - Load MNIST data (images of written digits)
 - Visualize some images
 - Build a multi-layer neural network
 - Train the neural network with stochastic gradient descent, dropout, and early stopping
 - Predict on new test images